

Software Testing HW2

312553027

1. add function

寫測試

建立以下測試來檢查 add 函數是否能正確加總兩個數字：

```
1 | test('adds two numbers', () => {
2 |     expect(Calc.add(2, 3)).toBe(5); // 2 + 3 = 5
3 | });
```

測試運行結果

運行測試時，得到以下結果，表示 add 函數的測試成功通過：

```
1 | > hw2@1.0.0 test
2 | > jest
3 |
4 | PASS ./Calc.test.js
5 |   ✓ adds two numbers (1 ms)
6 |
7 | Test Suites: 1 passed, 1 total
8 | Tests:       1 passed, 1 total
9 | Snapshots:   0 total
10 | Time:        0.18 s, estimated 1 s
11 | Ran all test suites.
```

2. subtract function

寫測試

建立 subtract 函數的測試，檢查兩個數字相減的結果是否正確：

```
1 | test('subtracts two numbers', () => {
2 |     expect(Calc.subtract(5, 2)).toBe(3); // 5 - 2 = 3
3 | });
```

測試運行結果

運行測試後，發現測試失敗，並報錯為 `TypeError: Calc.subtract is not a function`，這是由於尚未實現 subtract 函數。測試輸出如下：

```

1  > hw2@1.0.0 test
2  > jest
3
4  FAIL ./Calc.test.js
5    ✓ adds two numbers (2 ms)
6    ✕ subtracts two numbers
7
8    • subtracts two numbers
9
10   TypeError: Calc.subtract is not a function
11
12     6 |
13     7 | test('subtracts two numbers', () => {
14   > 8 |   expect(Calc.subtract(5, 2)).toBe(3); // 5 - 2 = 3
15       |                                   ^
16     9 | });
17
18     at Object.subtract (Calc.test.js:8:15)
19
20 Test Suites: 1 failed, 1 total
21 Tests:      1 failed, 1 passed, 2 total
22 Snapshots:  0 total
23 Time:       0.343 s, estimated 1 s
24 Ran all test suites.

```

實現 subtract 函數

為 Calc Class 新增 subtract 函數，以便正確執行數字相減：

```

1  class Calc {
2    static add(a, b) {
3      return a + b;
4    }
5    // 新增 subtract 函數
6    static subtract(a, b) {
7      return a - b;
8    }
9  }

```

測試運行結果

修改後再次運行測試，此時測試成功通過：

```

1 | > hw2@1.0.0 test
2 | > jest
3 |
4 | PASS ./Calc.test.js
5 |   ✓ adds two numbers (1 ms)
6 |   ✓ subtracts two numbers
7 |
8 | Test Suites: 1 passed, 1 total
9 | Tests:      2 passed, 2 total
10 | Snapshots:  0 total
11 | Time:       0.298 s, estimated 1 s
12 | Ran all test suites.

```

3. multiply function

寫測試

建立 multiply 函數的測試，檢查 multiply 函數是否能正確地進行數字相乘：

```

1 | test('multiplies two numbers', () => {
2 |   expect(Calc.multiply(2, 3)).toBe(6); // 2 * 3 = 6
3 | });

```

測試運行結果

運行測試後，發現測試失敗，並報錯為 `TypeError: Calc.multiply is not a function`，這是由於尚未實現 multiply 函數。測試輸出如下：

```

1 | > hw2@1.0.0 test
2 | > jest
3 |
4 | FAIL ./Calc.test.js
5 |   ✓ adds two numbers (1 ms)
6 |   ✓ subtracts two numbers
7 |   ✕ multiplies two numbers
8 |
9 |   • multiplies two numbers
10 |
11 |   TypeError: Calc.multiply is not a function
12 |
13 |     10 |
14 |     11 | test('multiplies two numbers', () => {
15 | > 12 |   expect(Calc.multiply(2, 3)).toBe(6); // 2 * 3 = 6
16 |         |                               ^
17 |     13 | });
18 |
19 |     at Object.multiply (Calc.test.js:12:15)
20 |
21 | Test Suites: 1 failed, 1 total
22 | Tests:      1 failed, 2 passed, 3 total
23 | Snapshots:  0 total
24 | Time:       0.286 s, estimated 1 s
25 | Ran all test suites.

```

實現 multiply 函數

為 Calc Class 新增 multiply 函數，以便正確執行數字相乘：

```
1  class Calc {
2      static add(a, b) {
3          return a + b;
4      }
5      static subtract(a, b) {
6          return a - b;
7      }
8      // 新增 multiply 函數
9      static multiply(a, b) {
10         return a * b;
11     }
12 }
```

測試運行結果

修改後再次運行測試，此時測試成功通過：

```
1  > hw2@1.0.0 test
2  > jest
3
4  PASS ./Calc.test.js
5    ✓ adds two numbers (2 ms)
6    ✓ subtracts two numbers
7    ✓ multiplies two numbers
8
9  Test Suites: 1 passed, 1 total
10 Tests:       3 passed, 3 total
11 Snapshots:   0 total
12 Time:        0.267 s, estimated 1 s
13 Ran all test suites.
```

4. divide function

寫測試

建立 divide 函數的測試，檢查 divide 函數是否能正確地進行數字相除：

```
1  test('divides two numbers', () => {
2      expect(Calc.divide(6, 3)).toBe(2); // 6 / 3 = 2
3  });
```

測試運行結果

運行測試後，發現測試失敗，並報錯為 `TypeError: Calc.divide is not a function`，這是由於尚未實現 divide 函數。測試輸出如下：

```

1  > hw2@1.0.0 test
2  > jest
3
4  FAIL ./Calc.test.js
5    ✓ adds two numbers (1 ms)
6    ✓ subtracts two numbers
7    ✓ multiplies two numbers (1 ms)
8    ✕ divides two numbers
9
10   • divides two numbers
11
12   TypeError: Calc.divide is not a function
13
14     14 |
15     15 | test('divides two numbers', () => {
16   > 16 |     expect(Calc.divide(6, 3)).toBe(2); // 6 / 3 = 2
17       |                               ^
18     17 | });
19
20     at Object.divide (Calc.test.js:16:15)
21
22   Test Suites: 1 failed, 1 total
23   Tests:       1 failed, 3 passed, 4 total
24   Snapshots:   0 total
25   Time:        0.29 s, estimated 1 s
26   Ran all test suites.

```

實現 divide 函數

為 Calc Class 新增 divide 函數，以便正確執行數字相除：

```

1  class Calc {
2    static add(a, b) {
3      return a + b;
4    }
5    static subtract(a, b) {
6      return a - b;
7    }
8    static multiply(a, b) {
9      return a * b;
10   }
11   // 新增 divide 函數
12   static divide(a, b) {
13     return a / b;
14   }
15 }

```

測試運行結果

修改後再次運行測試，此時測試成功通過：

```
1 | > hw2@1.0.0 test
2 | > jest
3 |
4 | PASS ./Calc.test.js
5 |   ✓ adds two numbers (1 ms)
6 |   ✓ subtracts two numbers
7 |   ✓ multiplies two numbers
8 |   ✓ divides two numbers
9 |
10 | Test Suites: 1 passed, 1 total
11 | Tests:      4 passed, 4 total
12 | Snapshots:  0 total
13 | Time:       0.269 s, estimated 1 s
14 | Ran all test suites.
```

5. divide function 處理除以零的情況

寫測試

為了確保 divide 函數能夠處理除以零的情況，新增一個測試來檢查：

```
1 | test('divides by zero', () => {
2 |   expect(() => {
3 |     Calc.divide(6, 0);
4 |   }).toThrow("Cannot divide by zero"); // should throw an error
5 | });
```

測試運行結果

測試對 divide 函數在除以零的情況下沒有拋出錯誤，導致測試失敗，這是由於 divide 函數還沒正確處理除數為零的情況。測試輸出如下：

```

1  > hw2@1.0.0 test
2  > jest
3
4  FAIL ./Calc.test.js
5    ✓ adds two numbers (2 ms)
6    ✓ subtracts two numbers (1 ms)
7    ✓ multiplies two numbers
8    ✓ divides two numbers
9    ✗ divides by zero (1 ms)
10
11    • divides by zero
12
13    expect(received).toThrow(expected)
14
15    Expected substring: "Cannot divide by zero"
16
17    Received function did not throw
18
19      20 |     expect(() => {
20      21 |       Calc.divide(6, 0);
21      > 22 |     }).toThrow("Cannot divide by zero"); // should throw an error
22         |         ^
23      23 |   });
24
25      at Object.toThrow (Calc.test.js:22:6)
26
27  Test Suites: 1 failed, 1 total
28  Tests:      1 failed, 4 passed, 5 total
29  Snapshots:  0 total
30  Time:        0.199 s, estimated 1 s
31  Ran all test suites.

```

修改 divide 函數

在 divide 函數中加入邏輯來檢查除數是否為零，如果為零，則拋出錯誤：

```

1  class Calc {
2    static add(a, b) {
3      return a + b;
4    }
5    static subtract(a, b) {
6      return a - b;
7    }
8    static multiply(a, b) {
9      return a * b;
10   }
11   // 修改 divide 函數，處理除以零的情況
12   static divide(a, b) {
13     if (b === 0) {
14       throw new Error("Cannot divide by zero");
15     }
16     return a / b;
17   }
18 }

```

測試運行結果

修改後再次運行測試，此時測試成功通過：

```
1 | > hw2@1.0.0 test
2 | > jest
3 |
4 | PASS ./Calc.test.js
5 |   ✓ adds two numbers (2 ms)
6 |   ✓ subtracts two numbers (1 ms)
7 |   ✓ multiplies two numbers
8 |   ✓ divides two numbers
9 |   ✓ divides by zero (5 ms)
10 |
11 | Test Suites: 1 passed, 1 total
12 | Tests:      5 passed, 5 total
13 | Snapshots:  0 total
14 | Time:       0.194 s, estimated 1 s
15 | Ran all test suites.
```

6. divide function 回傳浮點數結果

寫測試

為了確保 divide 函數在除法運算時回傳的是浮點數而非整數，新增以下測試來檢查結果是否為正確的浮點值：

```
1 | test('divides and returns float', () => {
2 |   expect(Calc.divide(7, 2)).toBeCloseTo(3.5); // 7 / 2 = 3.5
3 | });
```

測試運行結果

運行測試時，得到以下結果，代表除法回傳浮點數的功能已正確實現：

```
1 | > hw2@1.0.0 test
2 | > jest
3 |
4 | PASS ./Calc.test.js
5 |   ✓ adds two numbers (2 ms)
6 |   ✓ subtracts two numbers
7 |   ✓ multiplies two numbers
8 |   ✓ divides two numbers (1 ms)
9 |   ✓ divides by zero (4 ms)
10 |   ✓ divides and returns float
11 |
12 | Test Suites: 1 passed, 1 total
13 | Tests:      6 passed, 6 total
14 | Snapshots:  0 total
15 | Time:       0.274 s, estimated 1 s
16 | Ran all test suites.
```


Final Version of Calc.js

```
1  class Calc {
2    static add(a, b) {
3      return a + b;
4    }
5    static subtract(a, b) {
6      return a - b;
7    }
8    static multiply(a, b) {
9      return a * b;
10   }
11   static divide(a, b) {
12     if (b === 0) {
13       throw new Error("Cannot divide by zero");
14     }
15     return a / b;
16   }
17 }
18
19 module.exports = Calc;
```

Screenshot showing that all tests pass

```
● (Software_Testing) linux@linuxdeMacBook-Pro hw2 % npm test

> hw2@1.0.0 test
> jest

PASS ./Calc.test.js
  ✓ adds two numbers (2 ms)
  ✓ subtracts two numbers
  ✓ multiplies two numbers
  ✓ divides two numbers (1 ms)
  ✓ divides by zero (4 ms)
  ✓ divides and returns float

Test Suites: 1 passed, 1 total
Tests:       6 passed, 6 total
Snapshots:   0 total
Time:        0.274 s, estimated 1 s
Ran all test suites.
```

以上開發過程中遵循了TDD的流程。首先為 add、subtract、multiply 和 divide 函數編寫了測試，這些測試在初期會失敗。接著會根據測試的需求撰寫對應的程式碼，使測試通過。

整個過程中，每個功能的實現都基於對應的測試，確保了功能的正確性。在此過程中，並不需要額外的重構，因為每個步驟的實現都在開發初期就已經符合需求和測試要求，最終所有測試都成功通過。